

ROC, PR Curves & Calibration, Worked Out by Hand

Threshold sweeps, trapezoidal AUC, and expected calibration error

A complete numerical walk-through of how a model's raw probability scores turn into the evaluation curves a course tells you to "always plot". We take *one* tiny scored set of six examples, sweep the decision threshold across every score, and watch the confusion-matrix counts (TP, FP, FN, TN) march from one corner of the plot to the other. From those counts we build the ROC curve, the precision-recall curve, the trapezoidal areas underneath them, and the expected calibration error — every number produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It fixes a scored set of six (`prob`, `label`) pairs — small enough that you can verify every cell of every confusion matrix by eye — and then computes, by hand, the three families of classifier diagnostics: the ROC curve and its AUC, the PR curve and its average precision, and the reliability of the probabilities themselves via ECE.

How to read this. Each curve gets its own worked *Mini-example* box isolating the one threshold step that matters, alongside the running six-point set. A threshold sweep is just "slide a cutoff from $+\infty$ down to 0 and re-count" — read this to see why the resulting curves are step functions and why their area has a clean trapezoidal formula.

Concepts covered, in order: the scored set and its confusion matrix · the threshold sweep · TPR and FPR · the ROC curve as a list of points · trapezoidal ROC-AUC · precision and recall · the PR curve and average precision · confidence bins, accuracy vs. confidence · expected calibration error (ECE) with 2 and 3 bins · picking an operating threshold.

1 The setup: a scored set small enough to count by hand

A trained classifier does not output a class — it outputs a *score*, a number in $[0, 1]$ that we *threshold* to get a decision. All of ROC, PR and calibration are just different ways of summarising the same raw list of (score, true-label) pairs. So we shrink to the smallest list that still exercises every mechanism: six examples, with both classes present and the scores deliberately *not* perfectly separated.

example	e_1	e_2	e_3	e_4	e_5	e_6
score p	0.9	0.8	0.7	0.4	0.3	0.1
label y	1	0	1	1	0	0

The labels contain $P = 3$ positives ($y = 1$, at scores 0.9, 0.7, 0.4) and $N = 3$ negatives ($y = 0$, at scores 0.8, 0.3, 0.1). The list is already sorted by score, which is all the ranking-based metrics ever care about. Notice the one “mistake” in the ranking: the negative e_2 at 0.8 outscores the positive e_4 at 0.4 — that single inversion is what will keep the AUC below 1.

Intuition

There are two completely different questions you can ask of these scores, and they need different tools. **Ranking quality:** “do positives generally score higher than negatives?” — this is threshold-free, and it is exactly what ROC-AUC measures. **Calibration:** “when the model says 0.8, is it right about 80% of the time?” — this cares about the actual numeric value of the score, not just its order. A model can be a perfect ranker ($\text{AUC} = 1$) yet badly calibrated, and vice versa. Keep the two questions separate; this document answers both for the same six numbers.

2 Step 1 — The threshold sweep: counting TP, FP, FN, TN

We decide *positive* when the score $p \geq t$ for a threshold t , and *negative* otherwise. As we lower t from $+\infty$ (predict nothing positive) down through each score, examples flip from predicted-negative to predicted-positive one at a time. At every threshold we count the four confusion-matrix cells:

$$\text{TP} = \#\{p \geq t, y = 1\}, \quad \text{FP} = \#\{p \geq t, y = 0\}, \quad \text{FN} = \#\{p < t, y = 1\}, \quad \text{TN} = \#\{p < t, y = 0\}.$$

Sweeping t through the seven breakpoints $+\infty, 0.9, 0.8, 0.7, 0.4, 0.3, 0.1$ gives:

t	TP	FP	FN	TN	FPR	TPR	precision	recall
$+\infty$	0	0	3	3	0.000	0.000	—	0.000
0.9	1	0	2	3	0.000	0.333	1.000	0.333
0.8	1	1	2	2	0.333	0.333	0.500	0.333
0.7	2	1	1	2	0.333	0.667	0.667	0.667
0.4	3	1	0	2	0.333	1.000	0.750	1.000
0.3	3	2	0	1	0.667	1.000	0.600	1.000
0.1	3	3	0	0	1.000	1.000	0.500	1.000

The two rates that build the ROC curve are

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

TPR (= recall = sensitivity) is the fraction of the 3 true positives we caught; FPR is the fraction of the 3 true negatives we wrongly flagged.

Mini-example — this operation in isolation

Take the row $t = 0.7$. The cutoff $p \geq 0.7$ flags e_1, e_2, e_3 (scores 0.9, 0.8, 0.7). Of those, e_1 and e_3 are truly positive and e_2 is not, so $\text{TP} = 2$, $\text{FP} = 1$. The one positive left behind is e_4 (score 0.4), so $\text{FN} = 1$; the negatives left behind are e_5, e_6 , so $\text{TN} = 2$. Then

$$\text{TPR} = \frac{2}{2+1} = 0.667, \quad \text{FPR} = \frac{1}{1+2} = 0.333.$$

That is one point $(\text{FPR}, \text{TPR}) = (0.333, 0.667)$ on the ROC curve — and also the point $(\text{recall}, \text{precision}) = (0.667, 0.667)$ on the PR curve.

3 Step 2 — The ROC curve and its trapezoidal AUC

Plot TPR (vertical) against FPR (horizontal), one point per threshold row. Reading the sweep table top-to-bottom traces the curve from $(0, 0)$ up to $(1, 1)$:

$$(0, 0) \rightarrow (0, 0.333) \rightarrow (0.333, 0.333) \rightarrow (0.333, 0.667) \rightarrow (0.333, 1) \rightarrow (0.667, 1) \rightarrow (1, 1).$$

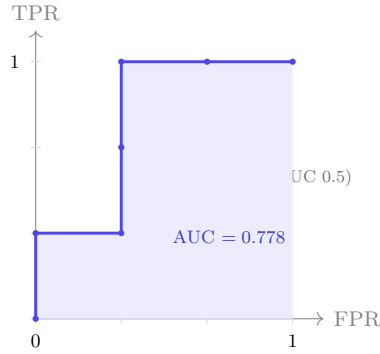
The **area under this curve (AUC)** is computed exactly by the *trapezoidal rule*: for consecutive points with horizontal gap $\Delta x = x_i - x_{i-1}$, the slab underneath has area $\Delta x \cdot \frac{1}{2}(y_i + y_{i-1})$. Summing the six slabs:

$$\begin{aligned} \text{AUC} &= \sum_i (x_i - x_{i-1}) \cdot \frac{1}{2}(y_i + y_{i-1}) \\ &= \underbrace{0 \cdot \frac{0+0.333}{2}}_0 + \underbrace{0.333 \cdot \frac{0.333+0.333}{2}}_{0.111} + \underbrace{0 \cdot \frac{0.667+0.333}{2}}_0 + \underbrace{0 \cdot \frac{1+0.667}{2}}_0 \\ &\quad + \underbrace{0.333 \cdot \frac{1+1}{2}}_{0.333} + \underbrace{0.333 \cdot \frac{1+1}{2}}_{0.333}. \end{aligned}$$

Vertical jumps (where $\Delta x = 0$) contribute zero area — only the horizontal moves build the AUC. The three non-zero slabs sum to

$$\text{ROC-AUC} = 0.111 + 0.333 + 0.333 = 0.778$$

(exactly $7/9 = 0.7778$). Above the diagonal 0.5 baseline, so the ranking is good but not perfect — the lone 0.8-negative-over-0.4-positive inversion costs us the missing $2/9$.



When this is the right choice

Reach for **ROC-AUC** when the two classes are roughly *balanced* and you care about overall ranking quality — “will a random positive be scored above a random negative?” (AUC is exactly that probability). Because FPR has the negative count $FP + TN$ in its denominator, a flood of easy true-negatives keeps FPR small, so ROC stays optimistic and stable under class balance. That is its strength when balanced — and its blind spot when positives are rare (next box).

4 Step 3 — The PR curve and average precision

The precision–recall curve plots precision against recall as the threshold drops. Using the same sweep, but indexed by the score breakpoints (precision is undefined at $t = +\infty$ where nothing is predicted positive):

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}$$

t	TP	FP	recall	precision
0.9	1	0	0.333	1.000
0.8	1	1	0.333	0.500
0.7	2	1	0.667	0.667
0.4	3	1	1.000	0.750
0.3	3	2	1.000	0.600
0.1	3	3	1.000	0.500

Precision is jagged (it dips at $t = 0.8$ when the negative e_2 sneaks in, then recovers) because each new prediction is either right or wrong, nudging the ratio up or down. **Average precision (AP)** summarises the curve as a recall-weighted mean of precision: at each step it multiplies the precision by the *increase* in recall, $\Delta R = R_k - R_{k-1}$ (with $R_0 = 0$):

$$AP = \sum_k (R_k - R_{k-1}) \cdot P_k.$$

Only the steps where recall actually increases contribute — recall climbs $0 \rightarrow 0.333$ (at $t=0.9$),

then $0.333 \rightarrow 0.667$ (at $t=0.7$), then $0.667 \rightarrow 1$ (at $t=0.4$); the others have $\Delta R = 0$:

$$\begin{aligned} \text{AP} &= \underbrace{(0.333 - 0) \cdot 1.000}_{0.333} + \underbrace{(0.667 - 0.333) \cdot 0.667}_{0.222} + \underbrace{(1.000 - 0.667) \cdot 0.750}_{0.250} \\ &= 0.333 + 0.222 + 0.250 = \boxed{0.806}. \end{aligned}$$

(Exactly $29/36 = 0.8056$.) Each surviving positive, taken in score order, “votes” the precision achieved at the moment it is recalled: 1.000 for e_1 , 0.667 for e_3 , 0.750 for e_4 , averaged $\frac{1}{3}(1.000 + 0.667 + 0.750) = 0.806$.

Watch out

The PR curve and AUC depend on the *positive rate*: the baseline (a useless random classifier) sits at precision = $P/(P+N)$, here 0.5, not at 0.5 area like ROC. So you *cannot* compare AP across datasets with different class balance, and you must not read $\text{AP} = 0.806$ as “0.806 as good as perfect” the way you can with AUC. Always report the positive rate alongside AP.

When this is the right choice

Prefer **PR-AUC** / **average precision** when positives are *rare* — fraud, disease, anomalies, retrieval. ROC’s FPR hides false positives behind a huge true-negative count, so a model that raises thousands of false alarms can still post a flattering ROC-AUC. Precision puts those false positives directly in the denominator ($\text{TP} + \text{FP}$), so the PR curve collapses honestly when the model floods you with false alarms. Rare positives $\Rightarrow$ trust PR.

5 Step 4 — Calibration: is a predicted 0.8 right 80% of the time?

ROC and PR only use the *order* of the scores. Calibration asks whether the score’s *value* is honest: among all examples scored near 0.8, is the empirical positive rate near 0.8? We measure the gap with the **Expected Calibration Error**: bin the scores by confidence, and in each bin compare the average confidence to the actual accuracy (here, the fraction of positives, since a high score predicts class 1):

$$\text{ECE} = \sum_b \frac{n_b}{N} | \text{acc}_b - \text{conf}_b |$$

where n_b is the bin’s count, $N = 6$ the total, conf_b the mean score in the bin, and acc_b the mean label in the bin.

Two equal-width bins $[0, 0.5)$ and $[0.5, 1]$. Bin 1 holds the low scores $\{0.4, 0.3, 0.1\}$ with labels $\{1, 0, 0\}$; bin 2 holds $\{0.9, 0.8, 0.7\}$ with labels $\{1, 0, 1\}$:

$$\begin{aligned} \text{bin } [0, 0.5) : \quad n_1 &= 3, \text{ conf}_1 = \frac{0.4+0.3+0.1}{3} = 0.267, \text{ acc}_1 = \frac{1}{3} = 0.333, |\cdot| = 0.067, \\ \text{bin } [0.5, 1] : \quad n_2 &= 3, \text{ conf}_2 = \frac{0.9+0.8+0.7}{3} = 0.800, \text{ acc}_2 = \frac{2}{3} = 0.667, |\cdot| = 0.133. \end{aligned}$$

$$\text{ECE}_2 = \frac{3}{6}(0.067) + \frac{3}{6}(0.133) = 0.033 + 0.067 = \boxed{0.100}.$$

Three equal-width bins $[0, \frac{1}{3})$, $[\frac{1}{3}, \frac{2}{3})$, $[\frac{2}{3}, 1]$ resolve the picture more finely:

bin	members (score)	n_b	conf_b	acc_b	$\frac{n_b}{N} \text{acc}_b - \text{conf}_b $
[0, 0.333)	0.3, 0.1	2	0.200	0.000	0.067
[0.333, 0.667)	0.4	1	0.400	1.000	0.100
[0.667, 1]	0.9, 0.8, 0.7	3	0.800	0.667	0.067

$$\text{ECE}_3 = 0.067 + 0.100 + 0.067 = \boxed{0.233}.$$

The number changed (from 0.100 to 0.233) when we re-binned — ECE is *bin-dependent*. The middle bin is the worst offender: the single example e_4 was scored 0.4 but is actually positive, a 0.6 miss that the coarse 2-bin grid averaged away.

Intuition

Picture the *reliability diagram*: plot acc_b against conf_b . A perfectly calibrated model lies on the 45° line $\text{acc} = \text{conf}$. ECE is just the average vertical distance from the dots to that line, weighted by how many examples each dot represents. Here the top bin sits *below* the diagonal ($\text{acc} 0.667 < \text{conf} 0.800$): the model is *overconfident* at the high end — it says 0.8 but is right only $\sim 67\%$ of the time.

Watch out

High AUC does not imply good calibration. A monotone re-scaling of the scores (e.g. squaring them) leaves the *ranking* — and therefore ROC-AUC and AP — completely unchanged, while wrecking the numeric values and blowing up ECE. If you need the probability itself (expected-cost decisions, thresholding by a business rule, ensembling), check ECE / a reliability diagram and apply temperature scaling or Platt scaling. And note: *none* of these curves picks a threshold for you. Reporting “AUC = 0.778” ships nothing — you still must choose an operating point (e.g. the threshold maximising F1, or the one meeting a recall floor) before the model decides anything.

6 The three diagnostics, at a glance

Diagnostic	Uses	Value here	Reads as
ROC-AUC	ranking only	0.778	P(pos ranked above neg); 0.5 = random
Average prec. (PR)	ranking, +ve rate	0.806	precision averaged over recall; baseline 0.5
ECE (2 bins)	score <i>values</i>	0.100	mean acc–conf gap, coarse bins
ECE (3 bins)	score <i>values</i>	0.233	same, finer bins (bin-dependent!)

The first two grade *how the model orders* examples; the last grades *whether its numbers mean what they say*. A deployment report needs all three plus a chosen operating threshold — not one headline figure.

7 Equation cheat-sheet

Predict positive	$p \geq t$ (sweep t over the distinct scores)
True/false positive rate	$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$
Precision / recall	$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}$
Trapezoidal AUC	$\sum_i (x_i - x_{i-1}) \frac{1}{2} (y_i + y_{i-1})$
ROC-AUC	area of (FPR, TPR) curve; 0.5 = random, 1 = perfect
Average precision	$\sum_k (R_k - R_{k-1}) P_k \quad (R_0 = 0)$
ECE	$\sum_b \frac{n_b}{N} \text{acc}_b - \text{conf}_b $
PR baseline	precision = $P/(P + N)$ (the positive rate)

Appendix A Reference implementation

Every number above is reproducible from the following NumPy program. It sweeps the threshold, builds the ROC and PR curves, integrates them with the trapezoidal rule, and computes ECE for any number of bins.

```
import numpy as np

# scored set: (prob, label)
probs = np.array([0.9, 0.8, 0.7, 0.4, 0.3, 0.1])
labels = np.array([1, 0, 1, 1, 0, 0])
P = labels.sum()          # 3 positives
N = (labels == 0).sum()   # 3 negatives

# --- threshold sweep: predict positive if prob >= t ---
scores = sorted(set(probs), reverse=True)      # 0.9, 0.8, 0.7, 0.4, 0.3, 0.1
roc = [(0.0, 0.0)]    # (FPR, TPR) starts at the origin
pr = []               # (recall, precision)
for t in scores:
    pred = (probs >= t).astype(int)
    TP = ((pred == 1) & (labels == 1)).sum()
    FP = ((pred == 1) & (labels == 0)).sum()
    FN = ((pred == 0) & (labels == 1)).sum()
    TN = ((pred == 0) & (labels == 0)).sum()
    TPR = TP / (TP + FN)
    FPR = FP / (FP + TN)
    prec = TP / (TP + FP)
    roc.append((FPR, TPR))
    pr.append((TPR, prec))    # recall == TPR

# --- trapezoidal ROC-AUC ---
xs = np.array([x for x, _ in roc])    # FPR ascending
ys = np.array([y for _, y in roc])    # TPR
auc = np.trapezoid(ys, xs)            # older numpy: np.trapz
print(f"ROC-AUC = {auc:.3f}")          # 0.778

# --- average precision (step rule, R_0 = 0) ---
ap, prev_r = 0.0, 0.0
for r, p in pr:
    ap += (r - prev_r) * p
    prev_r = r
print(f"AP = {ap:.3f}")                # 0.806

# --- expected calibration error ---
def ece(nbins, probs, labels):
    edges = np.linspace(0, 1, nbins + 1)
    total = 0.0
    for b in range(nbins):
        lo, hi = edges[b], edges[b + 1]
        m = (probs >= lo) & (probs <= hi if b == nbins - 1 else probs < hi)
        if m.sum() == 0:
            continue
        conf = probs[m].mean()          # average confidence in bin
        acc = labels[m].mean()          # empirical positive rate in bin
        total += (m.sum() / len(probs)) * abs(acc - conf)
    return total

print(f"ECE(2) = {ece(2, probs, labels):.3f}")    # 0.100
```

```
print(f"ECE(3) = {ece(3, probs, labels):.3f}") # 0.233
```

— end of document —